

Subversion (SVN) practical

David Henty, Adrian Jackson and Nick Brown

6 October 2016

1 Introduction

These practicals are designed to give some experience of using Subversion, (SVN) by going through the steps required to setup and use an SVN instance on your local machine. Often SVN is used remotely (i.e. the repository is based on a remote machine and you use SVN to upload and download data from it) but it can also be used locally (i.e. with the repository on the same machine that you are working on) and this mode of using SVN is useful for experimenting with, and learning, its functions and operations.

2 Create a Repository

The first thing that you will need to do is create an SVN repository to use. Ensure you are in your home directory and run the `svnadmin` command as follows:

```
$ svnadmin create svnrepos
```

Note that `svnrepos` in this example is the directory I want the SVN repository to be stored in. You do not have to call the directory `svnrepos`, you can call it whatever you like. Also, you do not have to create that directory prior to running the above command, SVN will create the directory for you.

You should also note that whilst this will work for the machine we are running this on there can be issues if your target for this command is a network mounted file system. This is fixed for Cirrus but using a network-mounted file system for your SVN repository storage on other machines is often best avoided.

3 Create your Project or Code Base

You have now created a working repository but it does not have any code within it. We will now add a project to the SVN. Creating a new project is done using the `svn import` command, but to create a new project you need to first create some existing structure. The convention for SVN is to use the following directory structure:

```
branches trunk tags
```

Create a directory in your account and add these directories to it, i.e.:

```
$ mkdir tmpsvn
```

```
$ cd tmpsvn
$ mkdir branches
$ mkdir tags
$ mkdir trunk
```

With these directory structures created you are ready to create your SVN project. All your data goes into the trunk directory. Run the `svn import` command from you `tmpsvn` directory as follows:

```
$ svn import -m "Initial aproject setup" file://$HOME/svnrepos/aproject
```

This will create a project called `aproject` in your SVN repository. Note that it has not added the current source code (i.e. your `tmpsvn` directory and everything in it) to SVN, it has only created the project in the repository. To actually use SVN to control this source you will have to check out the code (see the next part of the practical).

As with creating the SVN repository you should replace the `file:///` address used above with the location of the repository you used in the Create Repository section and you do not have to use `aproject` as the name of this SVN project, it can be anything you like.

The `-m "Initial aproject setup"` part of the import command provides a log message which SVN records as it creates the project. Most SVN commands that update the repository require a log message to be provided along with the update to record what operation has occur and provide details that could be useful to other using the repository.

Now the project has been added to SVN you can delete the `tmpsvn` directory.

4 Checkout a Working Copy

We now have a project created in SVN, but we do not currently have any source code under the control of SVN. We need to checkout a working copy of the project to be able to start developing source which is under SVN control.

Checkout a working copy of your new project using the following command (whereas before you will need to replace the `file:///` string with whatever your repository and project are called):

```
$ svn checkout file://$HOME/svnrepos/aproject
```

This will create a directory called `aproject` and add the `trunk`, `branches`, and `tags` directories to it. It will also report the current version of the repository along with the actions it has performed, which will probably look something like this:

```
A    aproject/trunk
A    aproject/branches
A    aproject/tags
Checked out revision 1.
```

We can see from the above information that SVN as added three directories (the A means added or created) and that the revision of the SVN repository is 1 (i.e. one change has happened to the repository since it was created).

If you enter the `aproject` directory and run the following command:

```
$ ls -la
```

You should see a `.svn` directory along with the other directories that have been created. This shows that this directory structure is under SVN control.

5 Adding files to the Working Copy

You can now add some content to your project and control it using SVN. Go into your `aproject/trunk` directory and add some new files using a text editor to add some arbitrary content. Once you have added the new files with some content you will have to tell SVN about them. If you run the `svn status` command in your `aproject/trunk` directory you will see that SVN reports them as not under version control (using the `?` character), i.e.

```
$ svn status
? mycode.c
? mycode2.c
```

To add these files to SVN you will need to use the `svn add` command. You can either specify each file you want to add individually, i.e.:

```
$ svn add mycode.c
A mycode.c
```

Or if you want to add all files in a directory you can use the `*` wildcard, i.e.

```
$ svn add *
A mycode.c
A mycode2.c
```

Now you have told SVN locally to add your files to SVN control but you need to commit this change to the repository before the changes you have made to your local copy are recorded in the SVN repository.

We will use the `svn commit` command to do this, but before you commit any new changes to a repository you should always check your code is up-to-date (i.e. check if any other changes have been added to the repository whilst you've been working). As you are working on your own repository which no one else is accessing in this example there will be no changes but it is always best practice to update your working copy before committing changes. To update your working copy run the `svn update` command, as follows:

```
$ svn update
```

Now, as there are no conflicting updates (i.e. no changes to the code we are working on) you can commit your changes as follows:

```
$ svn commit -m "Added two new files"
```

This will produce output like this:

```
Adding trunk/mycode.c
Adding trunk/mycode2.c
Transmitting file data ..
Committed revision 2.
```

As with the `svn import` command `svn commit` requires a message or comment to be provided using the `-m` flag.

6 Moving or Removing Files

All changes to file or directory structures within a working copy need to be done using SVN commands. Creating new files (as we did in the Adding files to the Working Copy section) does not require an SVN command initially, but deleting and moving files does as does creating and removing directories. The relevant commands are:

```
svn add

svn delete

svn move

svn copy

svn mkdir
```

As we saw when adding files any changes made using the above commands happen immediately in your working copy but need to be *committed* before they occur in the repository. For this exercise create a new directory in your trunk directory using the `svn mkdir` command, i.e.:

```
$ svn mkdir src
```

Now move one of your files into the new directory, i.e.:

```
$ svn move mycode2.c src/
```

You will see that SVN reports that it has added a file to the new directory and deleted the file in the current directory, i.e.

```
A src/mycode2.c
D mycode2.c
```

To commit these changes to the repository run `svn update` and then the `svn commit` command (providing an appropriate comment), i.e.:

```
$ svn update

$ svn commit -m "Create src directory and moved mycode2.c into it"
```

7 Modifications in more detail

Now edit your `src/mycode2.c` adding some arbitrary text to it. If we use the `svn status` command, then we can see this file has been changed.

```
$ svn status  
  
M src/mycode2.c
```

To view the modifications in further detail, you can use `svn diff`. Without any further arguments to show all differences, or with a filename to view differences made to one specific file.

```
$ svn diff  
  
Index: src/mycode2.c  
=====
```

---	src/mycode2.c	(revision 3)
+++	src/mycode2.c	(working copy)
@@	-1	+1,2 @@
	Existing contents	previously
	+Modified text	

If you wish to revert changes, then you can use the `svn revert` command to go back to the version in the `.svn` directory.

```
$ svn revert src/mycode2.c  
  
Reverted 'src/mycode2.c'
```

8 Resolving Conflicts

The last part of SVN functionality that we have not yet explored is resolving issues arising from multiple people editing the same file(s). This is exactly what SVN (and similar software) is created to help with but it is hard to replicate in a practical such as this (where you are the only person working on the repository). To duplicate such conflicts we will need to create two working copies and update both of them (no a scenario you will usually encounter with a SVN project).

In your home directory checkout a new working copy of your project, i.e.:

```
$ svn checkout file://$HOME/svnrepos/aproject aproject2
```

This will checkout a new working copy into a directory called `aproject2`.

Now alter the same file in both project and commit the changes from one of the projects. Now run `svn update` on the other project. You should be informed of conflicts, i.e.:

```
$ svn update  
  
Conflict discovered in 'mycode.c'.  
Select: (p) postpone, (df) diff-full, (e) edit,  
        (h) help for more options:
```

Now you will have to resolve the conflict. There are a number of ways to resolve conflicts; one is to produce a diff of the files (using the `df` command); one is to edit (using the `e` command) which will open the file in an editor with the conflict areas highlighted for you to manually fix (this will require the `EDITOR` environment variable to be correctly set or this option will not work); one is to postpone (using the `p` command) which allows you to resolve the conflict later (and creates a number of different files; a copy of the current file, a copy of the latest version from the repository, and a copy of the version that was checked out prior to the

changes that were made to the conflicted file); and finally you can choose to overwrite the repository version with your changes (using the mine-full `mf` command) or overwrite local changes with the repository version (using the theirs-full `tf` command).

When you edit or postpone conflict resolution files will be annotated with conflict markers, such as these:

```
<<<<<<< .mine
void main(){
    return;
=====
int main(){
    return 1;
>>>>>>> .r4
```

These show the lines that are different and state which version are from where (the `.mine` in this case is the local copy and `.r4` is the repository copy). To resolve this conflict you need to edit the file, decide on the changes that you want to happen, remove the conflict markers and then tell SVN that the conflict has been resolved using the `svn resolved` command, i.e.:

```
$ svn resolved mycode.c
```

Which will report something like this, and will also remove any temporary files that SVN created:

```
Resolved conflicted state of 'mycode.c'
```

Now you can commit you file as you would normally. Try the different mechanisms of conflict resolution by creating a conflict and then resolving it.

9 Find out more...

Subversion is a very popular revision control solution. There are plenty of tutorials and manuals available, as well as the inbuilt help, i.e. `svn help`.

A popular FAQ is available at <http://subversion.tigris.org/faq.html>, an online Subversion book at <http://svnbook.red-bean.com/>, and mailing lists at <http://subversion.tigris.org/mailling-lists.html>.

Eric Sink's Version control by example provides an acclaimed introduction to basic concepts, Subversion and Git: Sink, E. Version Control by Example. Pyrenean Gold Press, 25 July 2011. ISBN-10: 0983507902, ISBN-13: 978-0983507901. <https://www.amazon.com/Version-Control-Example-Eric-Sink/dp/0983507902/156-0246549-6231041>. Online at: <http://www.ericssink.com/vcbe/>.